University of Rwanda
College of Science and Technology
Computer and software engineering
Mobile application system and design
year 3
Reg: 223016911

on 27/02/2026

Assignment Q1 & Q2.

Question 1

State is any data in application that can change and affect what users see on the screen.

State management: is way we control, update and organize that changing data so the user interface updates correctly.

So, now explain different techniques:

Provider

provider allows us to store our app's state in one place and access or update it from anywhere in the widget tree without manually passing data down through constructors.

> How provider works:

1. State class (model): class extending ChangeNotifier) hold data and logic
2. notifyListener(): when data change it tell flutter to rebuild only the widget that depend on the data.
3. Consumer / context.watch(): widget listen to the provider and rebuild when the state changes.

> Why use provider, why it is usefull.

> Separates business logic from ui.

> Avoid props drilling (passing data manually through many widgets).

Riverpod

Riverpod is a modern and improved state management solution for flutter that helps manage application state in a safe, scalable and testable way without depending on BuildContext

> How provider works:

1. Create something called provider, which is holds and manages a piece of state. accessable any where in the application.
2. When a widgets needs data, it watches the provider to listen for changes
3. When state update, automatically rebuilds.

- Why Riverpod is powerful.
  - Compile - time safety
  - more scalable for large apps and better separation for logic and UI

Bloc

In Bloc, the UI does not change the state directly. Instead, it send an event, the bloc processes that event, and then emits a new state that updates the UI.

- How Bloc works
  1. the user performs an action.
  2. the UI send an event to the bloc.
  3. the Bloc processes the event using business logic
  4. the Bloc emits a new state
  5. the UI rebuilds based on new state.

  UI → Event → Bloc → state → UI.

- Why use Bloc.
  - Strong separation between UI and business logic
  - predictable and structured state flow.

Get x

Getx is a lightweight and powerful state management solution in Flutter that allows developers to manage state in a simple, reactive and effective way (UI updates automatically whenever the state changes).

- How Getx works.
  - State variable are made reactive.

  when the value of a reactive variable changes, only the widgets that depend on it build automatically.

- Why use Getx?
  - very simple and minimal code.
  - High performance.

Question 2

State management.	Small Apps	medium App	large / enterprise App	Team project	Fast Development	Strict Architecture
Set State.	✓				✓	
provider	✓	✓		✓	✓	
Riverpod	✓	✓	✓	✓		✓
Bloc		✓	✓	✓		✓
Getx.	✓	✓			✓	

CamScanner

Assignment

Benjamin **TUYISENGE** (**223016911**)

UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY Y3 Computer Engineering

Module: Mobile Application Systems and Design
Date: 27th Feb 2026

1. Provider in Flutter – Key Steps

1.1 Adding the Dependency

Before using Provider, add it to your pubspec.yaml:

```
#dependencies:
flutter:
sdk: flutter
provider: ^6.0.5 # check the latest version on pub.dev
```

Then run: bash command -> flutter pub get

1.2 Creating a State Class

Create a class to hold your data and logic.
It should **extend ChangeNotifier** so it can notify widgets when state changes.

```
import 'package:flutter/foundation.dart';

class CounterModel extends ChangeNotifier {
int count = 0;

void increment() {
count++;
notifyListeners(); // tells widgets to rebuild
}
}
```

- count is the state
- increment() updates state and triggers UI rebuild using notifyListeners()

1.3 Providing the State

Wrap your app (or part of the widget tree) with a **ChangeNotifierProvider** so the state is available to child widgets.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
runApp(
ChangeNotifierProvider(
create: (context) => CounterModel(),
child: MyApp(),
),
);
}
```

- create initializes the state
- All widgets under MyApp can access CounterModel

1.4 Accessing the State

Inside a widget, use **Consumer**, context.watch() or context.read() to get the state.

Using Consumer:

```
Consumer<CounterModel>(
```

```
builder: (context, counter, child) {
return Text("Count: ${counter.count}");
},
)
```

- Consumer listens for changes in CounterModel
- Rebuilds only this widget when state changes

1.5 Updating the State

Call methods on the state class to update it.

```
ElevatedButton(
onPressed: () {
context.read<CounterModel>().increment();
},
child: Text("Increment"),
)
```

- context.read() gets the state without listening for changes
- increment() updates state and triggers rebuild via notifyListeners()

1.6 How UI Rebuild Happens

1. State changes inside CounterModel via increment()
2. notifyListeners() tells Flutter widgets that depend on this state to rebuild
3. Only widgets that watch or consume the state are rebuilt
4. Other parts of the UI remain unaffected

Key Concept: This is efficient because only necessary widgets are rebuilt instead of the whole widget tree.